Home Gym Tracker

Product Requirements & Build Spec

Intern Handoff Document • June 2026 • v3

This document is your complete blueprint. Read it end to end before writing a single line of code.

2 Users & Profiles

Four members. Each has a profile with name, training frequency, injury/mobility notes, and full workout history. Profiles must be viewable and editable from within the app at any time.

Member	Frequency	Constraints	Notes
Mom	3x / week	Knee pain, elbow mobility	Avoid heavy squatting below parallel, skull crushers, and deep elbow flexion under load. Prefer hip-dominant lower body movements. Neutral grip pressing where possible.
Girlfriend	1x / week	None	No injury constraints. Wants to prioritize lower body. Single full-body session per week; keep it to 4-5 exercises and lean toward legs/glutes movements.
Brother	3x / week	None	No injury constraints. Standard 3-day split.
Coach (you)	3x / week	None	Primary app user. Builds and assigns workouts for others. Also logs own sessions.

Member management requirement

Dashboard needs an Add Member button. Each member card links to a profile view: injury notes (editable), training frequency, full session history expandable to set-level detail.

3 Equipment Inventory

All exercise suggestions must be filtered to this exact list. Every exercise in the library must be tagged with its required equipment. The app must also support adding new equipment through the UI as the gym grows.

Equipment	Details
Olympic Barbell	Standard 45 lb Olympic barbell
Bella Barbell	33 lb barbell — lighter option for beginners and accessory work
Dumbbells	Pairs from 5 lb to 50 lb in 5 lb increments
Squat rack	Adjustable j-hooks; dip bar and pull-up bar attached
Adjustable bench	Used for pressing (flat, incline, decline), supported rows, hip thrusts
Resistance bands	4 varieties: Orange (light), Red (medium), Blue (heavy), Green (very heavy)
Dip bar	Dip bar attachment on squat rack for dips, assisted dips, L-sits
Pull-up bar	Pull-up bar attachment on squat rack for pull-ups, chin-ups, and hanging exercises

Olympic rings	Hang from squat rack for ring push-ups, ring rows, ring dips, ring support holds
Bodyweight	Floor space for push-ups, lunges, planks, etc.

Add equipment from the UI

New equipment added here becomes immediately available as a tag on exercises — no code change needed.

4 Equipment Types & Movement Tracking

4.1 Equipment type lives on the set, not the exercise

Many movements can be done with different equipment. Bench press with a barbell vs. dumbbells is the same pattern — one exercise name, not three.

Equipment type is stored per set:

- Exercise: Bench Press — equipment type stored per set, not per exercise

- Set 1: 40 lb, Dumbbell, 8 reps
- Set 2: 45 lb, Dumbbell, 8 reps
- Set 3: 50 lb, Dumbbell, 8 reps

4.2 Cross-equipment historical data

The Prev column always pulls the most recent logged sets for that exercise, regardless of what equipment was used. The prior equipment type is shown next to the weight.

Example display when last session used dumbbells but today they are switching to barbell:

Set	Weight (today)	Reps	Prev (last session)

Set 1	___	8	40 lb — Dumbbell
Set 2	___	8	45 lb — Dumbbell
Set 3	___	8	50 lb — Dumbbell

Do not normalize across equipment types

Never convert weights between equipment types. Dumbbells are harder at the same number due to stabilization demands. Show raw history with the equipment type labeled — the coach decides.

4.3 Resistance band naming

Bands use the full color name. When Band is selected, also select color:

- Band — Orange (light)
- Band — Red (medium)
- Band — Blue (heavy)
- Band — Green (very heavy)

Store as band_orange, band_red, band_blue, band_green. Display the full label in the UI.

5 Feature Requirements

5.1 Dashboard

Feature	Description
Member selector	Cards for each member. Tapping a card scopes the dashboard to that member.
Add member	Button to create a new member profile: name, weekly frequency, injury/ mobility notes.
Member profile view	Tapping a member's name opens their profile: injury notes (editable), training frequency, and full session history in reverse chronological order. Each session is expandable to show exercises and per-set detail including equipment type.

Muscle heatmap	Grid of 8 muscle groups. Uses a red-to-white color scale based on weighted set count: primary muscle sets count as 1.0, secondary muscle sets as 0.5. Darkest red = 6+ weighted sets this week. White = zero sets. A muscle hit indirectly through compound movements still registers color. Gives the coach an immediate read on what needs attention.
Alert banner	Auto-generated nudge when a muscle group has not been trained in 4+ days.
Weekly stats	Sessions this week, muscles neglected, days since last session.

5.2 Workout Builder

Feature	Description
Workout generator	Generates a workout for a selected member + target muscle groups, respecting constraints and filtering to available equipment.
Equipment type per set	Each set row has a weight input (max 3 characters) and a separate equipment type selector. Full names used throughout: Olympic Barbell, Bella Barbell, Dumbbell, Band (with color), Bodyweight. Equipment type can vary between sets in the same exercise.
Previous weights	Each set row shows the weight and equipment type from the most recent logged session for that exercise. See Section 4.2.
Set completion	Checkbox per set. Marks done and highlights row green.
Add / remove sets	Add set button per exercise. Sets can be individually removed.
Add exercise	Manual entry: name, primary muscle group, rep target, compatible equipment tags. New exercises are saved to the library for future use.
Add equipment	Accessible from the workout builder and from a settings screen. Adds a new equipment type to the system, making it available as a tag on exercises.
Remove exercise	Trash icon removes an exercise from the active workout.
Swap exercise	Opens a panel of alternatives targeting the same primary muscle group, filtered to available equipment and the member's constraints.

5.3 Session Logging

- Saves each exercise with all set weights, equipment types, reps, and completion status
- Records timestamp and member ID

- Updates that member's muscle group coverage for the heatmap
- Logged sets become the Prev reference in the next workout

5.4 History View

Reverse-chronological list of past sessions per member. Each entry shows date and muscle groups hit. Expandable to full set detail including equipment type per set.

6 Data Model (PostgreSQL)

Raw SQL only — no ORM. Write queries directly so you understand exactly what the database is doing.

members

- id SERIAL PRIMARY KEY
- name VARCHAR(100) NOT NULL
- frequency_per_week INTEGER
- injury_notes TEXT
- created_at TIMESTAMPTZ DEFAULT NOW()

equipment

- id SERIAL PRIMARY KEY
- name VARCHAR(100) NOT NULL UNIQUE -- e.g. 'Olympic Barbell', 'Band - Orange'
- equipment_type VARCHAR(50) -- barbell, dumbbell, band, bodyweight, other
- notes TEXT

Normalizes equipment into its own table. Adding a new row here makes it immediately available as a tag on exercises.

exercises

- id SERIAL PRIMARY KEY
- name VARCHAR(150) NOT NULL
- primary_muscle_group VARCHAR(50)
- secondary_muscle_groups TEXT[] -- see explanation below
- notes TEXT

What secondary_muscle_groups is for

Every compound exercise works more than one muscle. A barbell row primarily trains the back, but the biceps and rear delts work hard too. primary_muscle_group is used for the heatmap and for the workout generator when you ask for 'a back exercise.' secondary_muscle_groups is a PostgreSQL array that stores everything else the exercise trains. This lets you answer questions like: 'Has the bicep been indirectly worked this week even if no direct curl was programmed?' It also prevents the generator from stacking too many exercises that hit the same secondary muscles, which would overtrain that group without the coach realizing it. You will not need this on day one, but design the schema for it now so you do not have to migrate later.

exercise_equipment (join table)

- exercise_id INTEGER REFERENCES exercises(id)
- equipment_id INTEGER REFERENCES equipment(id)
- PRIMARY KEY (exercise_id, equipment_id)

Links exercises to the equipment they require. A ring row needs Olympic Rings. A pure bodyweight exercise needs no row.

exercise_alternatives (join table)

- exercise_id INTEGER REFERENCES exercises(id)
- alternative_id INTEGER REFERENCES exercises(id)
- PRIMARY KEY (exercise_id, alternative_id)
- Bidirectional: if A is an alternative to B, store both (A,B) and (B,A)

workouts

- id SERIAL PRIMARY KEY
- member_id INTEGER REFERENCES members(id)
- date DATE NOT NULL
- notes TEXT

workout_exercises

- id SERIAL PRIMARY KEY
- workout_id INTEGER REFERENCES workouts(id)
- exercise_id INTEGER REFERENCES exercises(id)
- order_index INTEGER

sets

- id SERIAL PRIMARY KEY
- workout_exercise_id INTEGER REFERENCES workout_exercises(id)
- set_number INTEGER NOT NULL
- reps INTEGER
- weight NUMERIC(6,2) -- lb value. NULL for pure unloaded bodyweight.

- equipment_id INTEGER REFERENCES equipment(id) -- which specific equipment was used
- completed BOOLEAN DEFAULT FALSE

Why equipment_id is on sets, not exercises

A coach might warm up with the Bella Barbell and use the Olympic Barbell for working sets. Storing equipment at the set level captures that. At the exercise level, it would be lost.

7 Tech Stack

Layer	Choice	Why
Frontend	React + TypeScript	Industry standard. TypeScript enforces the shape of your API responses at compile time and catches entire categories of bugs before they reach production.
Styling	Tailwind CSS	Fast to iterate. Utility classes work well on tablet layouts without context switching between files.
Backend	Node.js + Express	Lightweight REST API. Same language as the frontend. Easy to reason about and debug.
Database	PostgreSQL	Relational databases are what most professional engineering teams use. Learning PostgreSQL properly here means you will be immediately productive in most companies you join.
DB access	Raw SQL (pg driver)	No ORM. Write queries directly with the 'pg' npm package. You will understand every query you write, which is exactly what interviewers test.
Testing	Playwright	End-to-end browser testing. Your strongest asset given your QA background. Tests run against a real browser and cover every critical user flow. This is code you will be very confident talking about in interviews.
API testing	Postman	Test every API endpoint manually before writing Playwright tests. Build a Postman collection with one request per endpoint, save example responses, and use it to verify behaviour during development. It also becomes useful documentation.
Code repository	GitHub	Every feature branch, PR before merge to main, commit messages that explain why.
CI/CD	GitHub Actions	Runs your Playwright tests automatically on every pull request. If tests pass, deploys to the server. You write the workflow file yourself so you understand every step.
Hosting	Linux VM (manual)	Spin up the frontend and backend manually first, then automate. This teaches you what deployment actually means at the infrastructure level.

Your QA background is a genuine asset here

Most junior devs ship without tests. You have QA training — use it. Being able to say 'I wrote end-to-end tests that run on every PR and gate deployment' is a real differentiator.

8 Testing with Playwright

Playwright controls a real browser programmatically — it opens the app, clicks buttons, fills forms, and asserts outcomes, headlessly, on every push.

8.1 What to test

Flow	What the test should verify
Add member	Fill in name, frequency, notes. Save. Member appears in dashboard selector with correct name.
Build a workout	Select member, pick muscle groups, generate workout. Exercises appear. Equipment type selector is present on each set row.
Log a session	Enter weights per set. Mark sets complete. Tap Log Session. Dashboard heatmap updates to reflect the muscle groups just trained.
Previous weights	Log a session. Start a new workout for the same member with the same exercise. Prev column shows the weights just logged with correct equipment type.
Swap exercise	Tap swap on an exercise. Alternative list appears. Select one. Card updates to the new exercise name.
Add exercise	Use the Add Exercise form. Fill in name, muscle group, reps, equipment. Save. Exercise appears in the library and is selectable in the builder.
Add equipment	Add a new equipment item. Verify it appears in the equipment type selector when building a workout.
Member profile	Tap a member name. Profile view opens. Injury notes are visible and editable.

Tests live in /tests and run with npx playwright test. GitHub Actions runs them on every pull request — if any fail, the PR is blocked and deploy does not happen. This is a quality gate, a concept from your QA background that applies directly here. Use your instincts: test edge cases (empty name field, negative weight), and never trust client input on the server — your cybersecurity background is the right lens for that.

9 Key SQL Patterns to Know

Understand each of these before you write it.

Fetching previous set data for an exercise

Given a member and an exercise, find the most recent session where that exercise was logged and return all its sets with equipment names.

```
SELECT s.set_number, s.weight, s.reps, e.name AS equipment_name
FROM sets s
JOIN workout_exercises we ON s.workout_exercise_id = we.id
JOIN workouts w ON we.workout_id = w.id
JOIN equipment e ON s.equipment_id = e.id
WHERE w.member_id = $1 AND we.exercise_id = $2
  AND w.date = (
    SELECT MAX(w2.date) FROM workouts w2
    JOIN workout_exercises we2 ON we2.workout_id = w2.id
    WHERE w2.member_id = $1 AND we2.exercise_id = $2
  )
ORDER BY s.set_number;
```

Correlated subquery: the inner SELECT finds the date of the most recent workout for that member/exercise pair. The outer query fetches all sets from that workout. Know this line by line.

Weighted set count per muscle group (heatmap)

For a given member, how many days ago was each primary muscle group last trained?

```
SELECT sg.muscle_group,
  SUM(CASE WHEN sg.is_primary THEN 1.0 ELSE 0.5 END) AS weighted_sets
FROM workouts w
JOIN workout_exercises we ON we.workout_id = w.id
JOIN sets s ON s.workout_exercise_id = we.id
JOIN (
  SELECT exercise_id, primary_muscle_group AS muscle_group, TRUE AS is_primary
  FROM exercises
  UNION ALL
  SELECT exercise_id,
    UNNEST(secondary_muscle_groups) AS muscle_group,
    FALSE AS is_primary
  FROM exercises
  WHERE secondary_muscle_groups IS NOT NULL
) sg ON sg.exercise_id = we.exercise_id
WHERE w.member_id = $1
  AND w.date >= CURRENT_DATE - INTERVAL '7 days'
GROUP BY sg.muscle_group
ORDER BY weighted_sets DESC;
```

The subquery expands each exercise into primary rows (1.0) and secondary rows (0.5). Sum those over 7 days per muscle group. Clamp at 6, divide by 6, interpolate: white (#FFFFFF) at 0, deep red (#C0392B) at 1.

Exercises compatible with available equipment

Given a list of available equipment IDs, return exercises that can be performed.

```
SELECT DISTINCT ex.id, ex.name, ex.primary_muscle_group

FROM exercises ex

JOIN exercise_equipment ee ON ee.exercise_id = ex.id

WHERE ee.equipment_id = ANY($1::int[])

ORDER BY ex.primary_muscle_group, ex.name;
```

ANY($1::int[]) matches equipment_id against the passed array — how the generator filters to available equipment.

10 API Endpoint Reference

All responses return JSON. All POST/PUT bodies are JSON with Content-Type: application/json. Validate all inputs server-side before touching the database.

Method	Path	Description
GET	/api/members	List all members
POST	/api/members	Create a new member
GET	/api/members/:id	Get member profile with injury notes
PUT	/api/members/:id	Update member profile and notes
GET	/api/members/:id/dashboard	Heatmap data + weekly stats
GET	/api/members/:id/history	Past sessions reverse chronological
GET	/api/equipment	List all equipment
POST	/api/equipment	Add a new piece of equipment
GET	/api/exercises	Full library — filterable by muscle_group and equipment_id
POST	/api/exercises	Add a new exercise to the library
GET	/api/exercises/:id/alternatives	Swap options for an exercise
POST	/api/workouts	Create a new workout for a member
GET	/api/workouts/:id	Full workout with exercises and sets
POST	/api/workouts/:id/log	Log a completed workout — writes all set data

GET	/api/members/:id/prev-sets/:exerciseId	Previous set weights + equipment for one exercise
POST	/api/workouts/generate	Generate a workout: member ID + target muscle groups
GET	/api/health	Health check — returns 200 OK

11 Build Phases

Work in phases. Do not move on until the current phase works end to end.

Phase 1 — Data foundation (Week 1)

1. Set up the GitHub repo. Establish the folder structure: client/, server/, sql/, tests/
2. Write the schema as raw SQL migration files in sql/migrations/. Run them locally.
3. Seed: 4 members, full equipment list, 30+ exercises tagged with muscle groups and equipment, alternative exercise mappings
4. Build and test all API endpoints using the pg driver. No ORM. Test each endpoint in Postman as you build it and save the requests as a collection in the repo.
5. Write at least one Playwright test per critical API flow

Phase 2 — Core UI (Week 2)

6. Member selector, dashboard screen (heatmap can use static data for now)
7. Add member flow and member profile/notes view
8. Workout builder: exercise cards, per-set weight inputs with full-name equipment type selector, set completion checkboxes
9. Add exercise, remove exercise, swap exercise flows
10. Add equipment flow

Phase 3 — Logging & history (Week 3)

11. Log Session — write all set data to the database
12. Fetch previous set data with equipment name and display in Prev column
13. Session history view per member, expandable to set detail

Phase 4 — Intelligence & polish (Week 4)

14. Dynamic muscle heatmap from real session data using the weighted set count query
15. Alert banner logic: 4+ days without a muscle group
16. Workout generator: member + muscle groups -> filtered, constraint-aware exercise list

17. iPad layout QA on a real device or browser emulator

Phase 5 — Deployment (Week 5)

18. Provision a Linux VM (Ubuntu 22.04). Install Node.js, PostgreSQL, nginx, pm2.
19. Manually deploy backend: clone repo, install deps, run migrations, seed, start with pm2
20. Manually deploy frontend: npm run build, serve with nginx, proxy /api/* to Express
21. Verify app is accessible at public IP
22. Write GitHub Actions workflow: run Playwright tests on every PR, deploy on merge to main
23. Store SSH key and server IP as GitHub repository secrets — never hardcoded
24. Write DEPLOYMENT.md documenting every manual step

12 Interview Talking Points

Be ready to explain every decision without hesitation.

On the database

Why PostgreSQL? It is the most common relational database in professional environments. The data has clear relationships — members, workouts, exercises, sets — and a relational model enforces those with foreign keys. I also wrote raw SQL instead of using an ORM, which means I can explain exactly what any query does.

On the data model

Why is equipment type on the set and not the exercise? Because you can use different equipment for different sets in the same session. A warm-up with the Bella Barbell and working sets with the Olympic Barbell would be lost if equipment was stored at the exercise level.

On secondary muscle groups

Why is that in the schema if you are not using it yet? Because compound movements train more than one muscle group. Designing for it now means I do not have to migrate later. It also lets me eventually prevent the generator from accidentally stacking too many exercises that hit the same secondary muscles.

On testing

I wrote Playwright end-to-end tests that run on every pull request and gate deployment. Given my QA background, I treated quality as a first-class concern — not an afterthought. Tests cover logging a session, adding a member, swapping exercises, and verifying the heatmap updates correctly.

On deployment

I did not use Heroku or a platform. I provisioned a Linux VM, installed and configured nginx and pm2 manually, then wrote a GitHub Actions workflow to automate it. I can explain what each piece is doing and why.

On security habits

All SQL uses parameterized queries — never string concatenation with user input. All inputs are validated server-side. Error responses never expose stack traces. These are habits from my cybersecurity background that I applied directly.

On scaling

If this needed to support 500 gyms, I would add a gyms table and a gym_id foreign key on members. All queries would be scoped by gym. The equipment table might become partly gym-specific. The core schema holds up well.

A note from your mentor

You have two backgrounds most developers do not: QA and cybersecurity. That means you already think about failure modes, edge cases, and untrusted input. The best engineers think this way too. Your job is to channel that instinct into code. Ask me why on everything — not just how to make it work, but why we made the choice we made. That is what gets remembered in interviews.